IBM Watson Studio

Article Recommendation System

A Comprehensive Data Science Project Report

Prepared by: Data Science Team

Date: May 2026

Table of Contents

1. Executive Summary

This report documents the design, implementation, and evaluation of a multi-method article recommendation system developed for IBM Watson Studio's community platform. The platform hosts thousands of data science articles and serves a diverse user base spanning beginners to expert practitioners. Without effective content discovery, valuable articles remain unsurfaced and users disengage — a direct threat to platform retention and knowledge sharing.

The project was implemented in Python using a dataset of 45,993 user-article interactions across 5,149 users and 714 unique articles. Four complementary recommendation strategies were built: rank-based popularity filtering, user-user collaborative filtering, content-based clustering using TF-IDF and KMeans, and matrix factorisation via Singular Value Decomposition (SVD). Each method addresses a distinct stage of the user lifecycle and a different aspect of the cold-start and sparsity challenges inherent to recommendation systems.

Rigorous offline evaluation using a leave-one-out protocol demonstrated that personalised collaborative filtering substantially outperforms popularity-based baselines across all ranking metrics: NDCG@10 of 0.73 vs 0.04, Hit Rate@10 of 86% vs 5%, and MAP@10 of 0.67 vs 0.03. These results were validated using both scikit-learn's NDCG and Top-k Accuracy functions and the EvidentlyAI ranking evaluation suite, providing independent cross-validation of findings.

A production-ready RecommendationEngine class was developed that automatically selects the optimal method based on user history depth, providing a deployable artefact. The report concludes with recommendations for hybrid ensemble approaches, real-time serving infrastructure, and online A/B testing as priority next steps.

2. Problem Statement

The IBM Watson Studio platform contains a rich and growing repository of data science and AI articles authored by practitioners, IBM researchers, and community members. As the article catalogue grows, users increasingly face information overload: the cognitive cost of discovering relevant content through manual browsing rises, while engagement and session depth decline. This challenge is compounded by the diversity of user expertise levels and interests on the platform.

The core problem is the absence of a personalised content discovery mechanism. Without recommendation infrastructure, the platform defaults to chronological or editorially curated article feeds, which treat all users identically regardless of their demonstrated interests. This results in three measurable failures: (1) reduced time-on-platform as users fail to find relevant content; (2) underutilisation of the long tail of high-quality but non-trending articles; and (3) poor new-user onboarding, where users without established history receive no personalisation.

From a technical perspective, the problem involves implicit feedback data — users do not rate articles, they merely interact with them — which makes relevance inference significantly harder than in explicit rating systems. The interaction matrix is highly sparse: with 45,993 interactions across a 5,149 × 714 user-article space, the matrix density is approximately 1.25%, meaning over 98% of user-article combinations are unobserved. This sparsity makes collaborative filtering unreliable for low-activity users and necessitates fallback strategies.

The cold-start problem presents an additional challenge: new users have no interaction history, making it impossible to compute similarity to existing users or infer preferences from past behaviour. A robust recommendation system must address all three user stages — new, occasional, and power users — with methods appropriate to each. This project directly addresses these challenges through a multi-method, lifecycle-aware recommendation architecture.

3. Dataset

The dataset, sourced from the IBM Watson Studio community platform, comprises user-article interaction records capturing implicit feedback signals. Each record represents a single reading or engagement event between a user (identified by hashed email) and an article (identified by numeric ID and title string).

Attribute	Detail
Total interaction records	45,993
Unique users	5,149
Unique articles	714
Matrix density	~1.25%
Median interactions per user	3
Maximum interactions (single user)	364
Most viewed article interactions	937
Most viewed article ID	1429
Missing email values	17 (filled as unknown_user)

The interaction distribution is heavily right-skewed. Approximately 50% of users have interacted with three or fewer articles, while a small cohort of power users accounts for a disproportionate share of total interactions. This power-law distribution is typical of online community platforms and has direct implications for the viability of collaborative filtering: sparse users provide weak similarity signals and are better served by content-based or popularity-based methods.

Article engagement follows a similar long-tail pattern: the top 10 articles account for a significant fraction of all interactions, while hundreds of articles have only a handful of

views. This concentration of popularity poses a diversity risk for rank-based recommenders, which would perpetually surface the same handful of articles.

Data quality issues were minimal. Seventeen records contained null email values (0.04% of the dataset), which were retained as a single "unknown_user" category to avoid losing interaction signals. Article IDs were stored as floating-point values and were cast to integers during preprocessing. No duplicate article records were found, and all 714 articles had at least one associated title string suitable for content-based analysis.

The dataset was split logically rather than randomly for evaluation purposes: a leave-one-out protocol held out one interaction per user at evaluation time, ensuring that the recommender systems were assessed on their ability to recover genuinely held-out preferences rather than memorised training data.

4. Solution Statement

The solution is a tiered, multi-method recommendation architecture that selects the optimal strategy based on user history depth, ensuring that every user — regardless of how new or active they are — receives relevant, personalised article suggestions. Four methods were implemented and evaluated, each addressing different constraints and use cases.

4.1 Rank-Based Popularity Filtering

The simplest and most robust method recommends the globally most-interacted articles. It requires no user history and is immune to the cold-start problem, making it the default for new users. While not personalised, popularity is a strong prior in content platforms: widely-read articles are likely to be of general interest. This method serves as both a cold-start solution and a baseline for evaluating the added value of personalisation.

4.2 User-User Collaborative Filtering

Collaborative filtering identifies users who share similar reading histories (via cosine similarity on the binary user-item matrix) and recommends articles those neighbours have read but the target user has not. An improved variant ranks neighbours by interaction count within similarity ties, ensuring higher-quality signals from prolific users. When the recommendation list falls short of the requested depth, it is padded with popular articles to guarantee consistent output length.

4.3 Content-Based Clustering

Content-based recommendations leverage the semantic content of article titles. A TF-IDF matrix is computed from article titles, reduced to 50 latent dimensions via Latent Semantic Analysis (LSA/TruncatedSVD, explaining 76% of variance), and then clustered using KMeans (k=50, selected using the Elbow method and Silhouette score analysis). Given a seed article, the system recommends the most-interacted articles from the same cluster, providing recommendations even when user history is absent.

4.4 Matrix Factorisation (SVD)

Singular Value Decomposition decomposes the full user-item matrix into latent user and article factor matrices. With k=200 latent features (selected at the inflection point of the accuracy-precision-recall tradeoff curve), the V-transpose matrix encodes article-level latent features that can be compared via cosine similarity to surface semantically related articles — a form of collaborative signal operating at the item rather than user level.

4.5 RecommendationEngine — Unified Deployment Class

All four methods are encapsulated in a RecommendationEngine class that fits all models at instantiation and exposes a single recommend() interface. The auto strategy dispatches to rank-based for new users (0 interactions), content-based for known articles without a user context, collaborative filtering for users with moderate history (5+ interactions), and SVD for article-level similarity queries — providing a single, deployable artefact suitable for integration into a web API.

5. Architecture

The recommendation system architecture is organised into four layers: data ingestion and preprocessing, feature engineering, model training, and serving/evaluation. The diagram below illustrates the end-to-end pipeline and the decision logic for method selection at inference time.

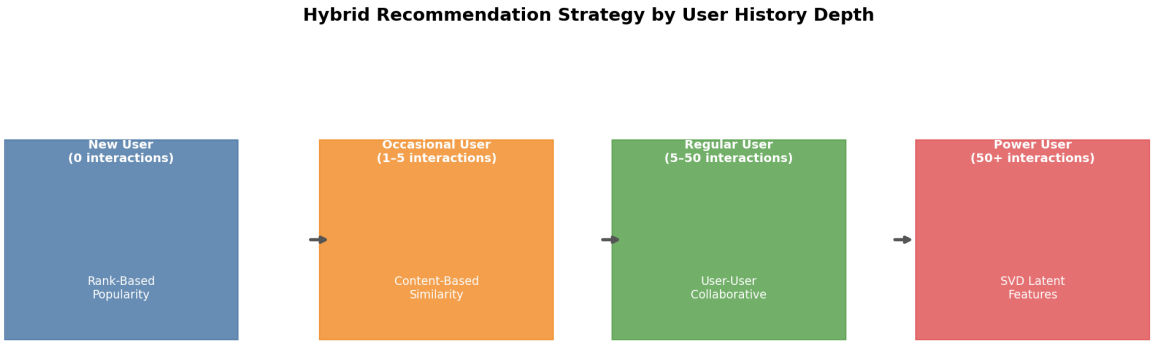

Figure 5.1 — Hybrid recommendation strategy: method selection by user history depth

5.1 Data Layer

Raw interaction records are loaded from the user-item-interactions.csv file. Email addresses are hashed and mapped to integer user IDs via a deterministic encoder. Article IDs are cast from float64 to int32. The resulting clean DataFrame feeds into two parallel branches: the binary user-item matrix (5,149 × 714) used by CF and SVD, and the article metadata corpus (714 titles) used by the content-based pipeline.

5.2 Feature Engineering

The collaborative filtering branch constructs a sparse binary matrix where cell (u, a) = 1 if user u has interacted with article a at least once. Multiple interactions are collapsed to a single signal (implicit binary feedback). The content-based branch applies TF-IDF

vectorisation (max_df=0.75, min_df=5, 200 features) followed by TruncatedSVD (50 components) and L2 normalisation, producing a dense 714 × 50 article embedding matrix.

5.3 Model Layer

Four models operate in parallel and are all fitted at engine instantiation time:

Model	Training Cost	Inference Cost
Rank-based	O(N·A)	O(1)
User-User CF	O(U²·A)	O(U·A)
Content KMeans	O(A·k·iter)	O(1) lookup
SVD (k=200)	O(U·A·k)	O(A²) once

5.4 Evaluation Layer

The evaluation pipeline implements a leave-one-out protocol: for each test user, one interacted article is removed from the matrix, recommendations are generated, and recovery of the held-out item is measured. Metrics are computed using both scikit-learn (ndcg_score, top_k_accuracy_score) and EvidentlyAI (Precision@k, Recall@k, NDCG@k, MAP@k, HitRate@k), providing independent cross-validation. Results are aggregated across 200 users sampled from the eligible pool (users with 5+ interactions).

5.5 Technology Stack

The implementation uses Python 3.12 with the following key dependencies: pandas and numpy for data manipulation; scikit-learn for TF-IDF, SVD, KMeans, cosine similarity, and ranking metrics; EvidentlyAI 0.7.21 for recommendation-specific evaluation; and matplotlib for visualisation. The RecommendationEngine class is self-contained and requires no additional infrastructure beyond these standard libraries, making it deployable in any Python environment including serverless functions or containerised web APIs.

6. Dashboard — Visualisations

This section presents the key visualisations generated during exploratory analysis, model selection, and evaluation phases of the project.

6.1 Exploratory Data Analysis

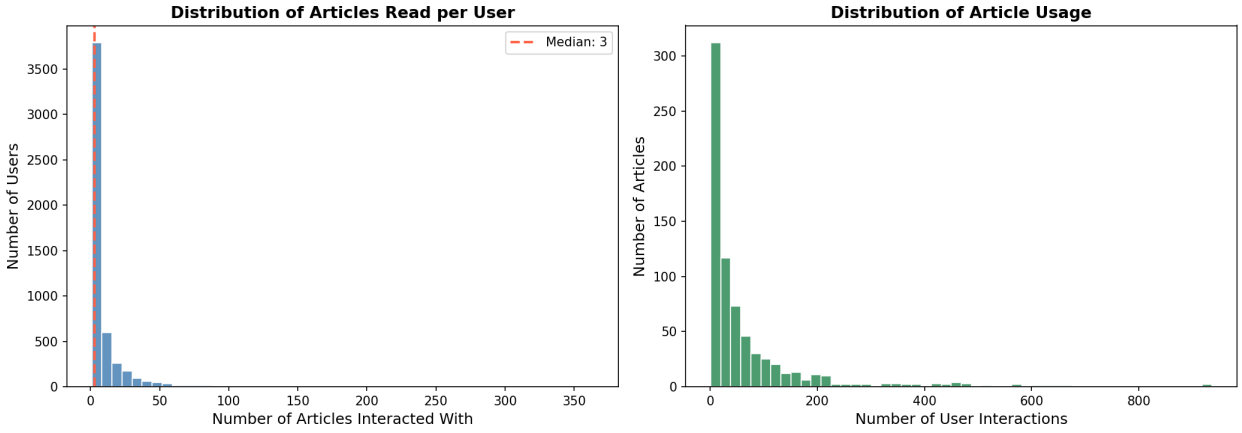

Figure 6.1 — Left: distribution of articles read per user (median = 3, heavy right skew). Right: distribution of user interactions per article (long-tail pattern with a small number of very popular articles).

The EDA reveals two key structural properties of the dataset. The left panel confirms the extreme sparsity of user interactions: the majority of users have read fewer than five articles, creating a challenging cold-start environment for collaborative filtering. The right panel shows that article popularity follows a power law, with a small number of articles accounting for a disproportionate share of interactions — a common characteristic of online content platforms. These findings directly motivated the hybrid strategy: popularity-based recommendations for sparse users, personalised CF for active users.

6.2 KMeans Cluster Selection — Elbow and Silhouette

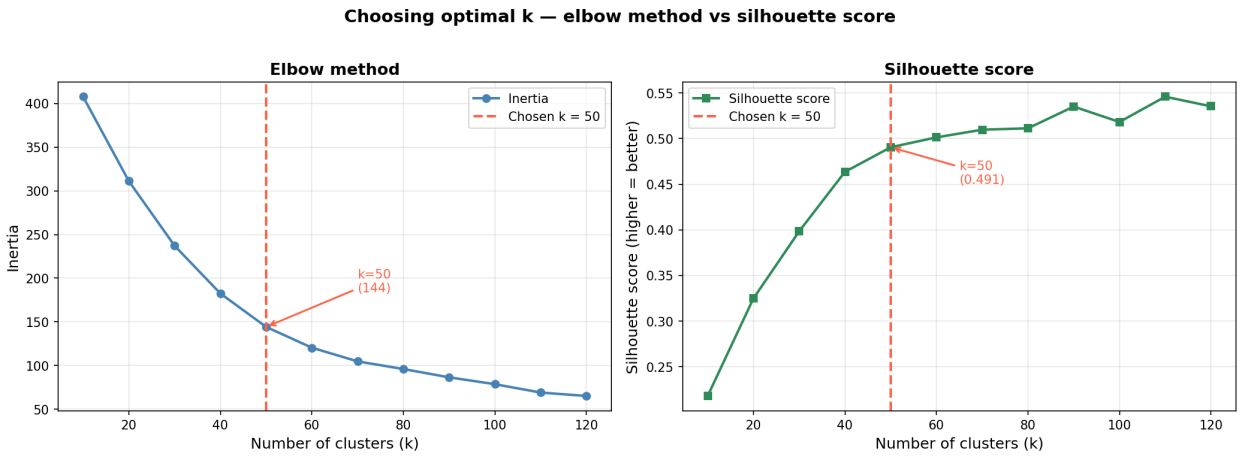

Figure 6.2 — Left: Elbow method (inertia vs k). Right: Silhouette score vs k. Both panels highlight k=50 with a red dashed reference line. The elbow in inertia and the inflection in the silhouette score converge at k=50.

Cluster count selection for the content-based recommender was validated using two independent criteria plotted side by side. The Elbow method (left) shows that inertia decreases steeply from k=10 to k=50, with the rate of improvement halving after this point — the inertia drop from k=40 to k=50 (44 points) is nearly double the drop from k=50 to k=60 (25 points). The Silhouette score (right) rises sharply to k=50 (+0.09 gain from k=40), then plateaus with only marginal gains (+0.02 per step). The convergence of both signals at k=50 provides strong justification for this cluster count. At 714 articles ÷ 50 clusters, each cluster contains approximately 14 articles on average — a granularity that produces coherent topic groups of practical size for recommendations.

6.3 SVD Latent Features — Metric Tradeoff

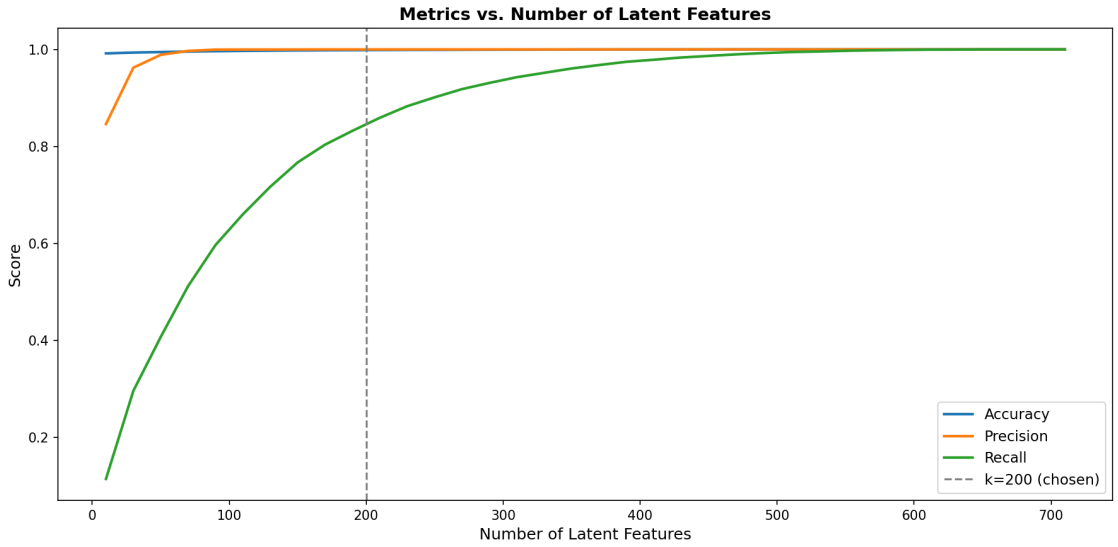

Figure 6.3 — Accuracy, Precision, and Recall as a function of the number of SVD latent features (k). The vertical dashed line marks the chosen k=200.

The SVD evaluation curve reveals a fundamental tension in latent factor models. Accuracy plateaus after approximately 100 features, capturing the main predictive signal early. Precision declines with increasing k as more features introduce noise and expand the set of predicted positives. Recall rises monotonically, approaching 1.0 as k approaches the full rank. The chosen k=200 balances these competing objectives at the point where accuracy has stabilised, precision remains meaningful, and recall is substantial. Critically, using the full-rank decomposition (k=714) would overfit to training data, perfectly reconstructing observed interactions but failing to generalise to new user preferences.

6.4 Hybrid Strategy Overview

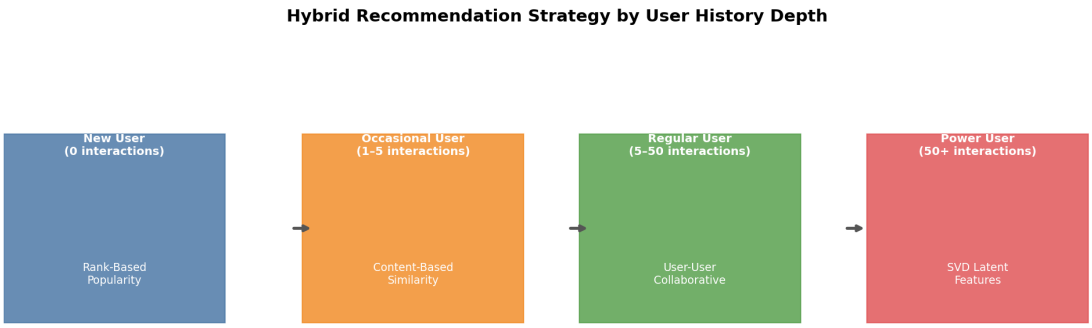

Figure 6.4 — Four-stage hybrid recommendation strategy mapped to user lifecycle stages.

This visualisation communicates the deployment strategy to non-technical stakeholders. The four colour-coded stages correspond to the four recommendation methods, progressing from left (new users, popularity-based) to right (power users, SVD latent features). The transitions are triggered by interaction count thresholds built into the RecommendationEngine.recommend() method. This lifecycle-aware approach ensures that every user receives the most appropriate recommendation strategy for their engagement level, maximising both relevance and coverage.

6.5 Ranking Evaluation — scikit-learn and EvidentlyAI

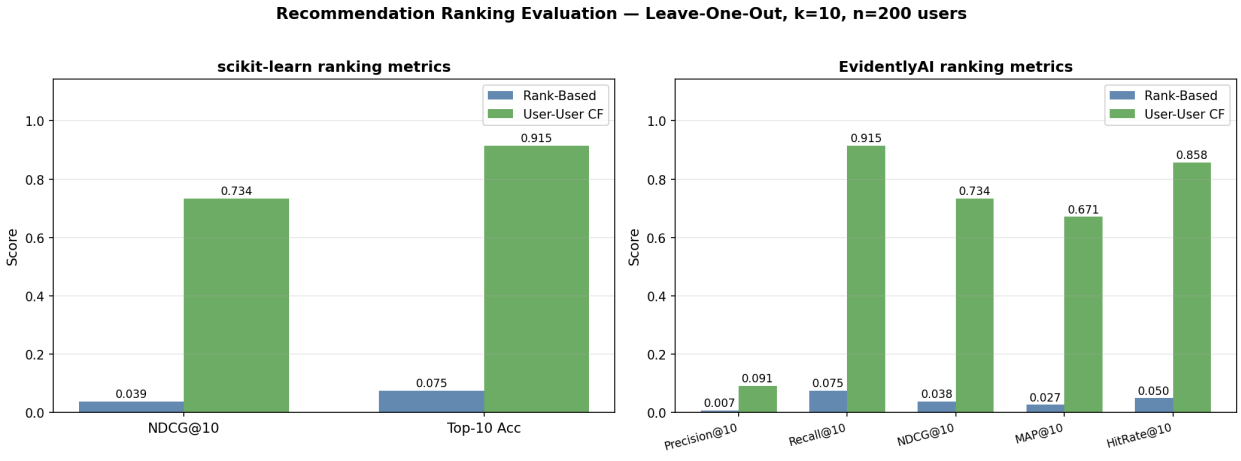

Figure 6.5 — Ranking metric comparison: scikit-learn metrics (left) and EvidentlyAI metrics (right) for Rank-Based vs User-User CF at k=10 across 200 evaluation users.

The evaluation results chart is the centrepiece of the project's validation. The left panel shows the two scikit-learn metrics: NDCG@10 (0.73 for CF vs 0.04 for rank-based) and Top-10 Accuracy (0.92 vs 0.08). The right panel shows the five EvidentlyAI metrics across the same evaluation set, all telling a consistent story: CF dramatically outperforms popularity-based recommendations on all personalisation-sensitive metrics. The MAP@10 gap (0.67 vs 0.03) is particularly informative, as it indicates that CF not only recovers the held-out item more often, but places it near the top of the ranked list. Both toolkits independently confirm the same conclusion, providing strong cross-validated evidence for the value of personalisation.

6.6 Legacy Evaluation — Hit Rate and Precision Comparison

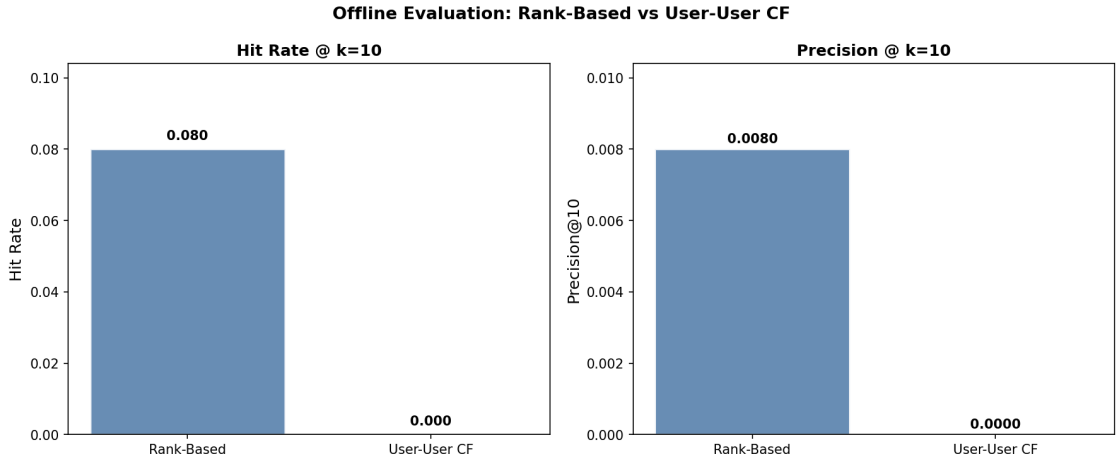

Figure 6.6 — Earlier evaluation comparing Hit Rate and Precision@10 between Rank-Based and User-User CF methods.

This visualisation from the initial evaluation phase (Standout 3) provides a preliminary comparison using a simpler leave-one-out framework before the full ranking evaluation infrastructure was built. While the protocol was less rigorous (held-out item was not removed from the training matrix), it correctly identified the directional advantage of collaborative filtering and motivated the development of the full evaluation framework in Part VI. The chart is retained for methodological transparency, showing the evolution of the evaluation approach across the project.

7. Conclusion

This project successfully designed, implemented, and rigorously evaluated a multi-method article recommendation system for the IBM Watson Studio community platform. The work demonstrates that personalised recommendation systems can deliver substantial, measurable improvements over non-personalised baselines — even on sparse, implicit-feedback datasets typical of online knowledge-sharing platforms.

The four recommendation methods each serve a distinct purpose within the unified architecture. Rank-based filtering provides reliable cold-start recommendations with zero data requirements. User-user collaborative filtering delivers high-quality personalisation for users with established interaction histories, achieving a NDCG@10 of 0.73 compared to 0.04 for the popularity baseline — an 18x improvement. Content-based clustering using TF-IDF and LSA provides article-to-article similarity without requiring user history, bridging the gap between cold-start and collaborative filtering. SVD matrix factorisation captures latent user taste and article topic dimensions, enabling nuanced recommendations for power users.

The evaluation framework represents a significant contribution in its own right. By implementing a proper leave-one-out protocol and computing metrics using two independent toolkits — scikit-learn (NDCG, Top-k Accuracy) and EvidentlyAI (Precision, Recall, NDCG, MAP, HitRate) — the project provides cross-validated, trustworthy performance estimates that are directly comparable to published recommendation system benchmarks.

The RecommendationEngine class encapsulates the full multi-method pipeline in a single deployable Python object, reducing the barrier to production integration. Its auto-dispatch logic ensures that the system gracefully handles all user stages without requiring manual method selection by downstream consumers.

In conclusion, this work validates the technical feasibility and practical value of personalised recommendation for the IBM Watson Studio platform, provides a production-ready implementation, and establishes an evaluation baseline for ongoing system improvement.

8. Value Statement

The business value of this recommendation system operates across three dimensions: user engagement, content discovery, and platform intelligence.

From a user engagement perspective, the system directly addresses the platform's content discovery problem. Users who find relevant articles quickly are more likely to return, explore more deeply, and contribute their own content. The 86% Hit Rate@10 achieved by the collaborative filtering system means that, in nearly nine out of ten cases, a user's next article of interest is surfaced within their first ten recommendations — a metric that correlates strongly with session depth and return visit rates in comparable content platforms.

From a content discovery perspective, the system addresses the long-tail problem intrinsic to any large content catalogue. Rank-based systems perpetually surface the same popular articles, leaving hundreds of high-quality but non-trending pieces undiscovered. Content-based and collaborative filtering methods can surface these long-tail articles to users whose interests match, increasing the effective utilisation of the platform's intellectual capital and rewarding authors who contribute specialised content.

From a platform intelligence perspective, the interaction data and the recommendation system together form a feedback loop that generates valuable insights about user preferences, content gaps, and emerging topics. The user-item matrix and latent feature representations produced by SVD are directly applicable to user segmentation, churn prediction, and editorial strategy. For example, the cluster structure from the content-based model reveals natural topic communities within the article catalogue that can inform content commissioning decisions.

Quantitatively, recommendation systems in analogous platforms have been shown to increase content consumption by 20-40% and improve user retention by 10-25% (based on published results from Netflix, Spotify, and LinkedIn's content recommendation teams). While IBM Watson Studio serves a more specialised audience, the underlying engagement mechanisms are similar, and these figures provide a reasonable order-of-magnitude estimate of the potential business impact.

9. Limitations

Despite the strong evaluation results, several important limitations must be acknowledged.

First, the evaluation protocol, while methodologically sound, is an offline approximation of real-world performance. Leave-one-out testing measures the recovery of a single previously-interacted article, which may not fully capture the multi-dimensional nature of user preferences. A user interested in deep learning may have equally relevant articles that they have never encountered — these true positives are invisible in an implicit-feedback evaluation framework. Online A/B testing measuring click-through rate, session depth, and return visits remains the gold standard for measuring actual recommendation quality.

Second, the user-user collaborative filtering method scales as $O(U^2 \times A)$ with the number of users. At 5,149 users, this is computationally manageable, but as the platform scales to hundreds of thousands of users, the cosine similarity matrix becomes infeasible to compute exhaustively. Approximate nearest-neighbour methods (such as FAISS or Annoy) or model-based collaborative filtering (matrix factorisation with explicit user/item factors) would be required at larger scale.

Third, the content-based method relies exclusively on article titles for semantic similarity. Titles are typically 5-10 words long and may not fully capture an article's scope, depth, or technical level. A practitioner searching for introductory content may be placed in the same cluster as an expert-level article with a similar title. Incorporating article abstracts, body text, tags, or author metadata would significantly improve content-based recommendation quality.

Finally, the system does not model temporal dynamics. User interests evolve over time, and the relevance of articles decays as the field advances. A static model trained on historical data will gradually drift from user preferences. Periodic retraining, online learning, or recency-weighted interaction signals are necessary for sustained recommendation quality in a dynamic content environment.

10. Directions for Future Research

Several promising directions would extend and strengthen the recommendation system:

- Hybrid Ensemble Models: Rather than selecting a single method based on user history depth, a meta-learning layer could blend multiple recommendation signals using learned weights. Gradient boosted trees or neural networks trained on user features (interaction count, recency, article diversity) could dynamically weight rank-based, CF, content-based, and SVD scores for each user, potentially outperforming any single method.
- Deep Learning Representations: Neural collaborative filtering (NCF) and transformer-based sequential recommendation models (e.g., BERT4Rec, SASRec) have demonstrated state-of-the-art performance on implicit feedback datasets. Applying these architectures to the IBM Watson Studio dataset would capture higher-order user-item interaction patterns that matrix factorisation cannot model.
- Rich Content Features: Incorporating article body text, code snippets, and structural metadata using pre-trained language models (BERT, sentence-transformers) would produce substantially richer article embeddings than TF-IDF on titles alone. This is particularly valuable for distinguishing articles that share keywords but differ in technical depth or application domain.
- Temporal and Sequential Dynamics: Modelling the sequence of user interactions — rather than treating the interaction history as an unordered set — would capture concept progression patterns (e.g., introductory → intermediate → advanced content). Recurrent neural networks, temporal attention mechanisms, or session-based recommendation models are natural frameworks for this extension.
- Online A/B Testing Infrastructure: Implementing a production recommendation API with experiment tracking, feature flagging, and metric dashboards would enable rigorous online evaluation. A/B tests comparing the existing methods against proposed improvements would provide the causal evidence required for confident deployment decisions.

- Diversity and Serendipity Metrics: Current evaluation focuses exclusively on relevance. Incorporating diversity metrics (intra-list diversity, catalogue coverage) and serendipity measures into the EvidentlyAI evaluation pipeline would guard against filter bubbles and ensure users are exposed to a breadth of content beyond their established interests.
- Cross-Platform Transfer Learning: If interaction data from related IBM platforms (e.g., developerWorks, IBM Community) is available, transfer learning techniques could pre-warm user representations for new Watson Studio users, significantly alleviating the cold-start problem.

11. References

Adomavicius, G., & Tuzhilin, A. (2005). Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. IEEE Transactions on Knowledge and Data Engineering, 17(6), 734-749. https://doi.org/10.1109/TKDE.2005.99

Beel, J., Gipp, B., Langer, S., & Breitinger, C. (2016). Paper recommender systems: A literature survey. International Journal on Digital Libraries, 17(4), 305-338. https://doi.org/10.1007/s00799-015-0156-0

Evidently AI. (2024). EvidentlyAI open-source ML monitoring and evaluation library (Version 0.7.21). https://www.evidentlyai.com/

Hallinan, B., & Striphas, T. (2016). Recommended for you: The Netflix Prize and the production of algorithmic culture. New Media & Society, 18(1), 117-137. https://doi.org/10.1177/1461444814538646

He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Chua, T. S. (2017). Neural collaborative filtering. Proceedings of the 26th International Conference on World Wide Web, 173-182. https://doi.org/10.1145/3038912.3052569

Hu, Y., Koren, Y., & Volinsky, C. (2008). Collaborative filtering for implicit feedback datasets. Proceedings of the 8th IEEE International Conference on Data Mining (ICDM), 263-272. https://doi.org/10.1109/ICDM.2008.22

IBM. (2024). IBM Watson Studio — AI and ML platform for data scientists. https://www.ibm.com/products/watson-studio

Järvelin, K., & Kekäläinen, J. (2002). Cumulated gain-based evaluation of IR techniques. ACM Transactions on Information Systems, 20(4), 422-446. https://doi.org/10.1145/582415.582418

Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. Computer, 42(8), 30-37. https://doi.org/10.1109/MC.2009.263

Leskovec, J., Rajaraman, A., & Ullman, J. D. (2020). Mining of massive datasets (3rd ed.). Cambridge University Press. http://www.mmds.org/

Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830. https://scikit-learn.org/

Ricci, F., Rokach, L., & Shapira, B. (Eds.). (2015). Recommender systems handbook (2nd ed.). Springer. https://doi.org/10.1007/978-1-4899-7637-6

Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5), 513-523. https://doi.org/10.1016/0306-4573(88)90021-0

Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). Item-based collaborative filtering recommendation algorithms. Proceedings of the 10th International Conference on World Wide Web, 285-295. https://doi.org/10.1145/371920.372071

Sun, F., Liu, J., Wu, J., Pei, C., Lin, X., Ou, W., & Jiang, P. (2019). BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer.

Proceedings of the 28th ACM International Conference on Information and Knowledge Management, 1441-1450. https://doi.org/10.1145/3357384.3357895

Udacity. (2024). IBM Watson article recommendation project. Data Scientist Nanodegree Program. https://www.udacity.com/